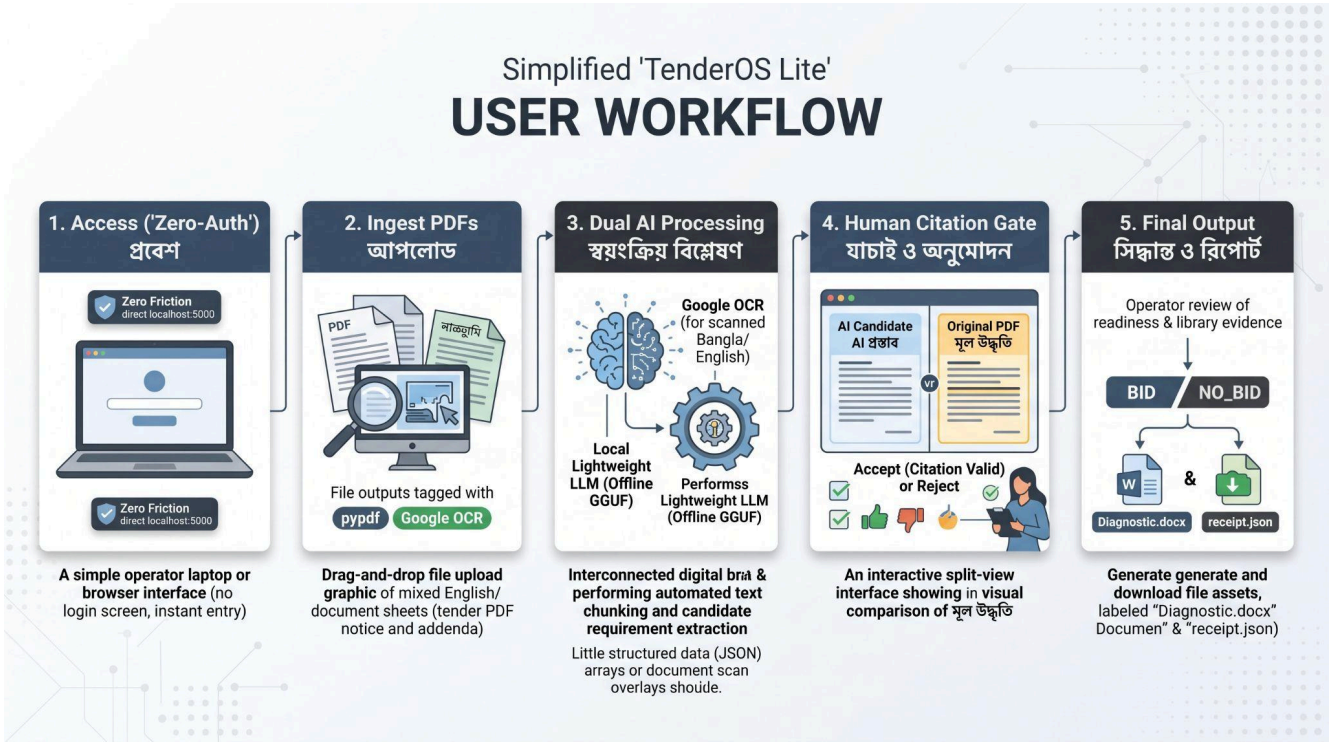
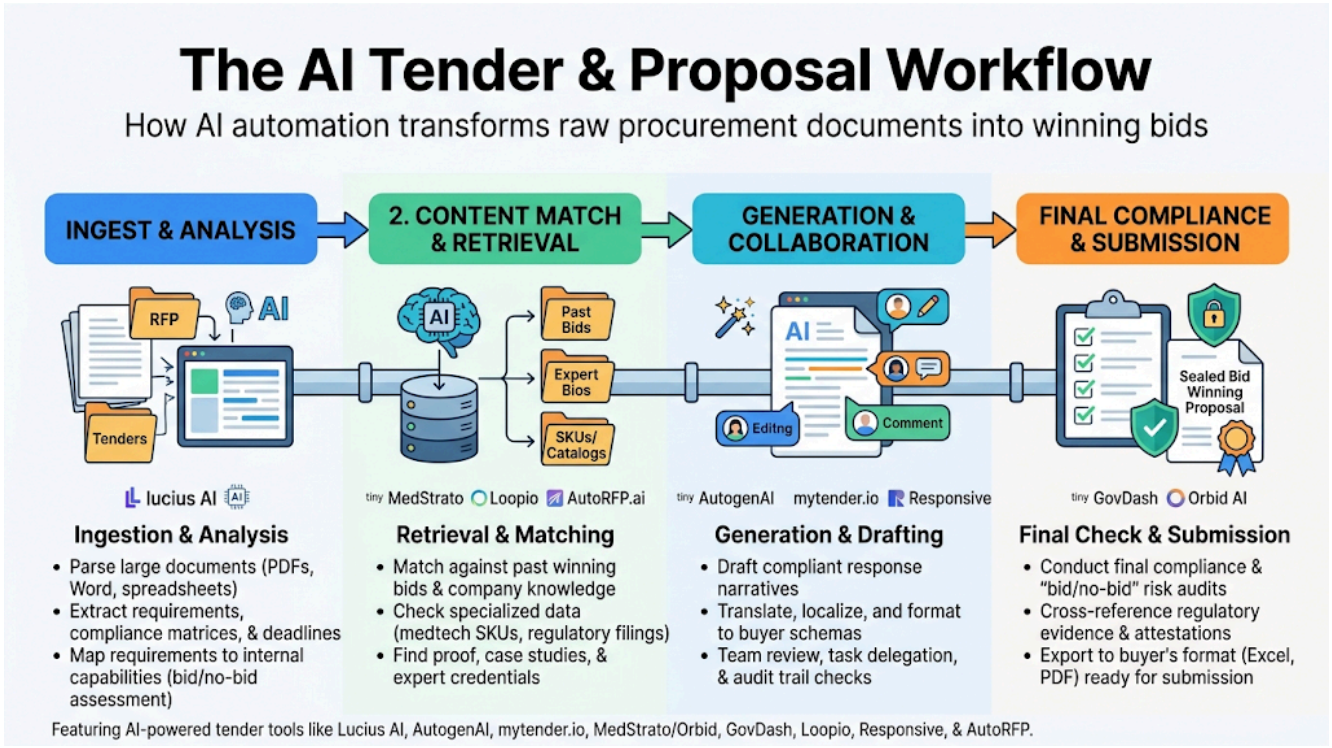


Competitors



Lucius AI (ailucius.com)

Lucius AI helps teams quickly figure out if a government contract is worth going after. Instead of writing the proposals, it reads through long PDFs and spreadsheets to pull out key requirements, deadlines, and potential deal-breakers. Everything is linked back to the exact page in the original file so you can double-check the details.

Process(i tested the demo):
https://drive.google.com/file/d/1MAdP_5edcFLywQjNejExmzklxdkB7DnO/view?usp=sharing

AutogenAI (autogenai.com)

AutogenAI acts as an intelligent writing assistant for larger companies and government contractors. It learns your brand's tone of voice and draws from past successful proposals to quickly generate outlines and draft responses, helping bidding teams work much faster.

Process: https://drive.google.com/file/d/1hR-xtqYKpd47l5xfKlahtAhSan5xJlDQ/view?usp=sharing

mytender.io (mytender.io)

mytender.io is designed for small and medium-sized businesses. It keeps all your company policies, case studies, and past proposals in one central place, using them to pull together first drafts in minutes. It also includes simple tools to assign tasks and track progress with your team.

Process: https://drive.google.com/file/d/1OFsl7S8KFp1ZiKjQwqbsoMD7zPL8c73m/view?usp=sharing

MedStrato (medstrato.com)

MedStrato is built specifically for medical device manufacturers. It compares product specs against hospital requirements, checks for safety and regulatory compliance, and highlights any missing info. Once you win a contract, it helps you keep track of ongoing commitments and renewal dates.

GovDash (govdash.ai)

GovDash focuses on businesses bidding for government contracts. It reviews official request documents to automatically build checklist spreadsheets, match past work samples, and draft responses that hit every requirement. It is built with high security standards so defense and civil contractors can work securely.

Loopio (loopio.com)

Loopio helps larger organizations answer questionnaires and proposal requests. It maintains a library of pre-approved company answers and automatically suggests matches for incoming questions. It connects with common tools like Word, Slack, and Salesforce so team members can easily collaborate and review answers.

Responsive (responsive.io)

Responsive (formerly RFPIO) helps manage the entire response lifecycle, from sales proposals to security assessments. It automatically populates answers, connects with workplace tools, and automates form fills on procurement sites. It also offers analytics to show what worked best across past bids.

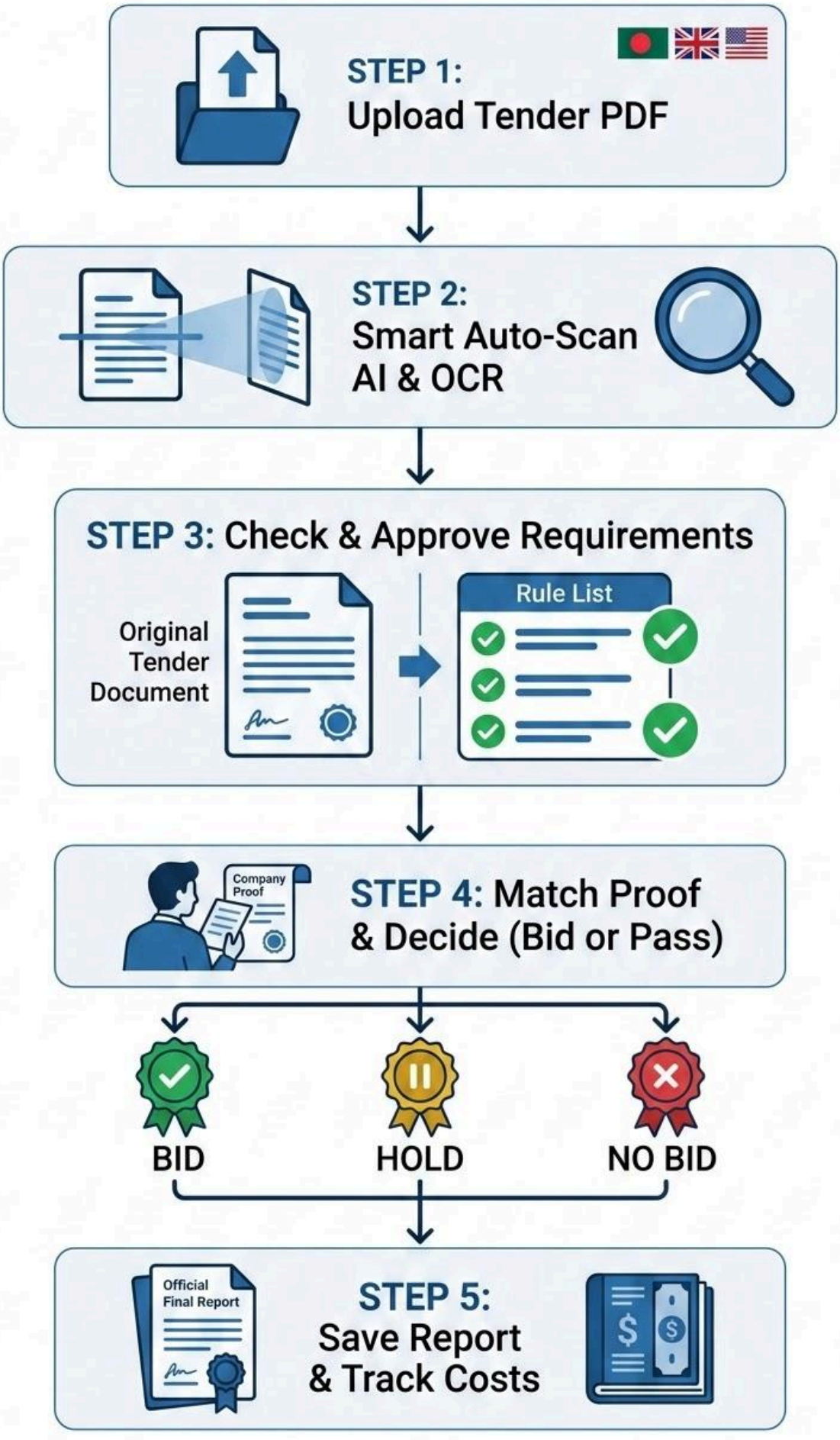
AutoRFP.ai (autorfp.ai)

AutoRFP.ai simplifies proposal writing by pulling information directly from your existing documents, like product sheets and past proposals. You don't need to organize a manual question-and-answer library. It quickly fills out detailed spreadsheets and forms, saving teams hours of copy-pasting.

Process: https://drive.google.com/file/d/1w5DwwWpELSK5vvPrsdI08osZjMKt5dKH/view?usp=sharing

Open Source Solutions

PrivateGPT on GitHub


STEP 1:
Upload Tender PDF
STEP 2:
Smart Auto-Scan
AI & OCR
STEP 3: Check & Approve Requirements
Original
Tender
Document
Rule List
Company
Proof
STEP 4: Match Proof
& Decide (Bid or Pass)
BID
HOLD
NO BID
Official
Final Report
STEP 5:
Save Report
& Track Costs

How It Works in 5 Easy Steps

1. **Upload Document**: You upload any tender document (even scanned Bengali paper PDFs) into TenderOS.
2. **Smart Reading**: The system automatically reads through every page and finds all mandatory rules, deadlines, and financial requirements. Every rule points back to the exact line in the paper so there are no mistakes or AI guesses.
3. **Quick Review**: You review the found rules side-by-side with the original paper and click to accept or reject them.
4. **Match Proof & Decide**: The system checks if your company has the required certificates and work experience on file. You then decide whether to **BID** (go for it), **HOLD** (pause), or **NO BID** (pass).

Final Summary & Expenses: Any costs or payments are tracked in a secure financial ledger, and an official final summary report is created for your team.

TenderOS Lite — Product Requirements Document (PRD)

Document ID: TOS-PRD-LITE-1.0

Project: TenderOS Simplified Architecture (TenderOS Lite)

Document Version: 1.0 (Production Release)

Stakeholders: Procurement Specialists, Bid Operations Managers, Technical Auditors

---

1. Vision & Strategic Rationale

TenderOS Lite strips away enterprise access complexity (SSO, JWT tokens, user databases, role tables) to deliver an uninhibited, appliance-style tender assessment workbench. While authentication has been excised, **zero functional compromise** has been made to the core engine: every document remains hash-verified, requirements are grounded in verified quotes, decisions enforce human accountability, and commercial ledgers remain strictly append-only.

By integrating **Google Cloud Vision OCR**, the platform overcomes the limitation of image-only and scanned Bengali tender notices commonly found on national procurement portals (such as Bangladesh e-GP). A **local open-source language model** ensures confidential eligibility documents never leak outside the operator's infrastructure while maintaining zero runtime API token costs.

---

2. Personas & Core User Journeys

2.1 The Sole Operator / Procurement Consultant

- **Goal:** Ingest a 150-page mixed English/Bangla tender schedule, isolate mandatory eligibility criteria within 15 minutes, verify corporate evidence coverage, record a BID decision, and issue an immutable client diagnostic report.
- **Pain Point Addressed:** Scanned Bengali PDFs previously resulted in manual transcription; high API billing risks on commercial models are neutralized by local offline execution.

2.2 User Journey Flow

1. **Direct Entry:** Operator opens browser (`localhost:5000`); no login barrier.

2. **Tender Ingestion:** Uploads tender PDF notice and addenda. System automatically computes SHA-256 hashes.

3. **Extraction & OCR:** Native text is parsed; scanned Bangla pages are sent to Google OCR. Local LLM extracts requirement candidates into structured JSON.

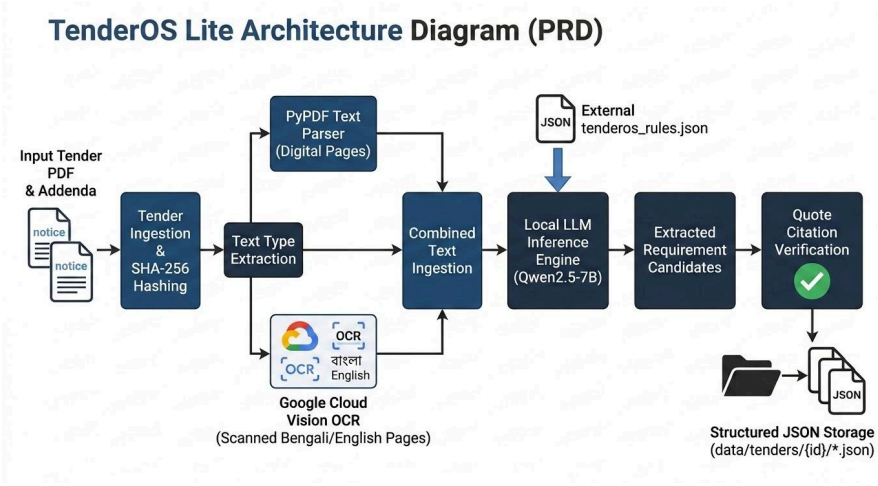
4. **Citation Review:** Operator reviews candidates side-by-side with original PDF citations; accepts verified requirements.

5. **Readiness & Evidence Match:** System matches requirements against the local evidence vault.

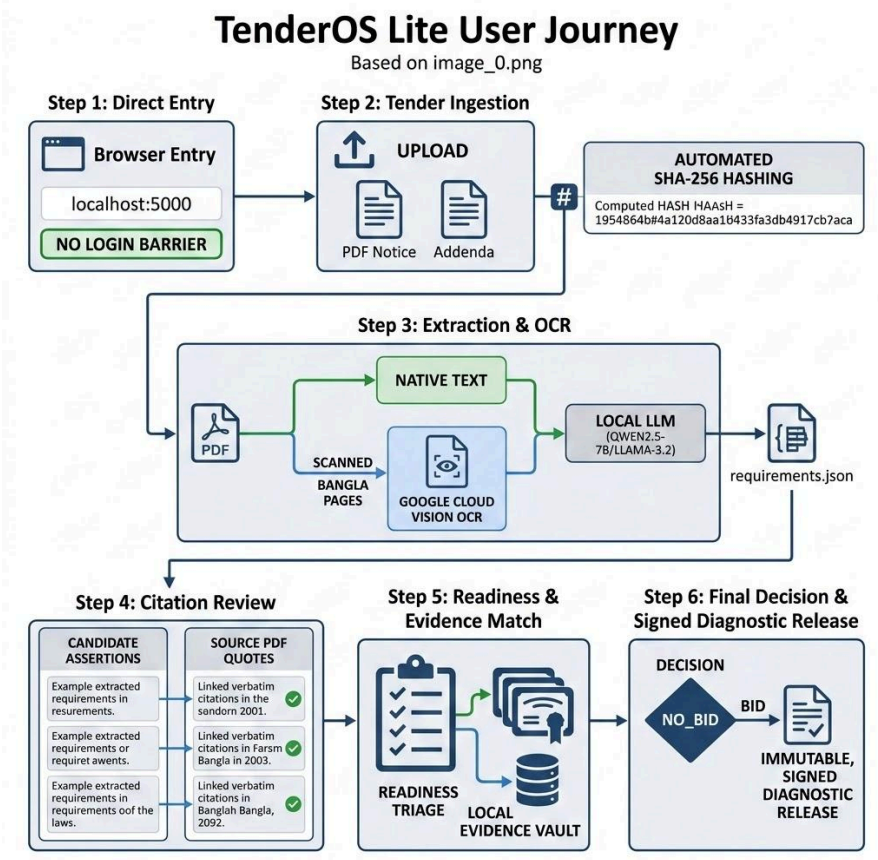
6. **Decision & Release:** Operator inputs BID/NO_BID decision and generates an immutable, signed diagnostic release.

Explain

System Architecture Flow



User Journey Flow



---

3. Detailed Product Capabilities

Module	Simplified Lite Architecture	Operational Value

Access Control	Open Intranet / Localhost (Zero Login, Zero RBAC)	Instant start; zero credential friction; zero token expiration during long review sessions.
Persistence Engine	Structured JSON Files (`data/tenders/{id}/*.json`)	Human-readable, transparent file trees; zero database service management; instant backups via file copy.
Bangla OCR Extraction	Hybrid: `pypdf` (digital) + Google Cloud Vision OCR (scanned)	High-accuracy extraction of complex Bengali typographic ligatures and tabular government schedules.
Inference Engine	Local Lightweight Model (Qwen2.5-7B-Instruct / Llama-3.2-3B)	100% offline document privacy; zero per-token cloud costs; predictable inference execution.
Prompt & Rule Controls	External `tenderos_rules.json` File	Procurement analysts can fine-tune scoring weights and extraction rules without modifying Python code.
Evidence & Integrity	SHA-256 Hash Chaining & Verbatim Quote Verification	Preserves forensic defense against hallucinations: no requirement is accepted without proof of quote.
Commercial Ledger	Append-Only JSON Ledger (BDT Minor Units)	Financial transparency; strict distinction between cash received and net margin; non-editable history.

---

4. Rule Engine Specification (`tenderos_rules.json`)

To prevent hardcoded logic, all business rules reside in a separate JSON file:

```
{
  "system": {
    "version": "1.0-lite",
    "currency_code": "BDT",
    "minor_unit_factor": 100
  },
  "extraction": {
    "ocr_provider": "google_cloud_vision",
    "ocr_languages": ["bn", "en"],
    "scanned_page_char_threshold": 40,
    "chunk_size": 800,
    "chunk_overlap": 100,
    "max_sources_per_tender": 20,
    "max_pages_per_tender": 600
  },
  "categories": [
    "eligibility",
    "technical",
    "financial",
    "documentary",
    "joint_venture",
```

```
    "litigation"
  ],
  "llm_inference": {
    "model_name": "qwen2.5:7b-instruct",
    "runtime": "ollama",
    "temperature": 0.0,
    "max_output_tokens": 2048,
    "enforce_json": true
  },
  "citation_verification": {
    "strict_whitespace_normalization": true,
    "fuzzy_similarity_threshold": 0.92
  },
  "commercial_ledger": {
    "allowed_kinds": ["RECEIPT", "DIRECT_COST", "REFUND", "REVERSAL"],
    "allow_negative_entries": false
  }
}
```

---

5. Success Metrics & Quality Targets

- **Extraction Accuracy:** ≥ 95% quote-exactness across both English and Bengali scanned tender documents.
- **Review Acceleration:** Reduce human tender readiness triage from 6 hours down to under 45 minutes.
- **Zero Hallucination Admission:** 100% rejection rate for LLM candidate assertions that lack valid text citations.
- **Audit Defensibility:** 100% deterministic re-verification of releases from stored raw source PDFs and JSON records.

TenderOS Lite — Technical Requirements & Design Document (TRD)

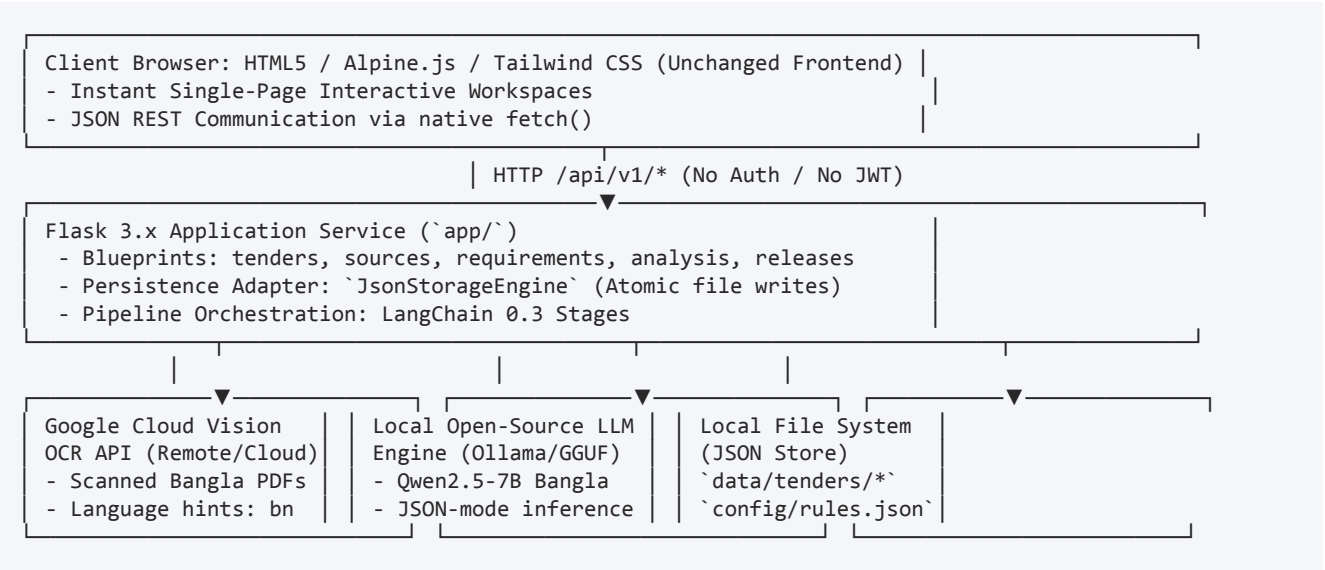
Document ID: TOS-TRD-LITE-1.0

Project: TenderOS Simplified Architecture (TenderOS Lite)

Implementation Reference: Flask 3.x / LangChain 0.3 / Google Cloud Vision OCR / Local LLM / JSON Storage / Alpine.js

---

1. Technical System Architecture



---

2. Directory & Repository Structure

```
tenderos-lite/
├── run.py                          # Flask bootstrap entrypoint
├── requirements.txt                # Pinned dependencies
├── config/
│   └── tenderos_rules.json         # Externalized business rules & thresholds
├── data/                           # All document and JSON persistence
│   ├── tenders/
│   │   └── {tender_id}/
│   │       ├── tender.json         # Master tender metadata
│   │       ├── inventory.json      # Agreed document inventory
│   │       ├── requirements.json   # Extracted and reviewed requirements
│   │       ├── evidence.json       # Private organizational evidence
│   │       ├── decisions.json      # Bid/No-Bid decision history
│   │       ├── ledger.json         # Append-only commercial transactions
│   │       ├── runs/               # Analysis runs & frozen receipts
│   │       ├── releases/           # Versioned diagnostic snapshots
```

```
│   │       ├── sources/            # Original uploaded PDF files
│   │       └── ocr_cache/          # Cached Google OCR page responses
│   └── audit/
│       └── system_audit.jsonl      # Append-only hash-chained audit log
├── app/
│   ├── __init__.py                 # Flask application factory
│   ├── config.py                   # System configuration & env bindings
│   ├── storage.py                  # Atomic JSON File Engine
│   ├── ocr_engine.py               # Google Cloud Vision Bengali/English OCR
│   ├── llm_engine.py               # Local LangChain LLM interface (Ollama/GGUF)
│   ├── pipelines/
│   │   ├── stages.py               # 5-stage bounded analysis sequence
│   │   ├── citation.py             # Quote & citation re-verification
│   │   └── prompts.py              # Strict JSON-structured prompt templates
│   ├── api/
│   │   ├── tenders_api.py          # CRUD tenders & inventory
│   │   ├── sources_api.py          # PDF upload, OCR execution
│   │   ├── requirements_api.py     # Requirements management & review
│   │   ├── analysis_api.py         # Run orchestration & progress polling
│   │   ├── decisions_api.py        # Decision recording & staleness
│   │   ├── ledger_api.py           # Commercial entries & totals
│   │   └── releases_api.py         # Snapshot generation & export
│   ├── templates/                  # Server-rendered Jinja2 HTML shells
│   └── static/js/                  # Alpine.js state managers & UI logic
```

---

3. Data Storage & Schema Design (`JsonStorageEngine`)

The system uses atomic file persistence. Updates read the JSON file, execute mutations in memory, write to a `.tmp` file, and replace the existing file atomically via `os.replace`.

3.1 `tender.json`

```
{
  "tender_id": "tnd-948194",
  "title": "Supply of High-Performance Hardware",
  "buyer": "Department of Public Works",
  "notice_id": "DPW/2026/G-04",
  "deadline": "2026-11-15T14:00:00+06:00",
  "currency": "BDT",
  "full_input_version": 3,
  "attestation_version": 1,
  "created_at": "2026-10-15T09:30:00Z"
}
```

3.2 `requirements.json`

```
[
  {
    "requirement_id": "req-001",
    "source_id": "src-sha256-abc...",
    "source_page": 12,
    "source_quote": "The Tenderer shall have a minimum of 5 years of overall experience.",
    "text": "Minimum 5 years general experience in hardware supply.",
    "category": "eligibility",
```

```
    "mandatory": true,
    "status": "HELD",
    "reviewed": true,
    "extraction_method": "LLM_V2",
    "created_at": "2026-10-15T10:00:00Z"
  }
]
```

3.3 `ledger.json`

```
[
  {
    "entry_id": "led-01",
    "kind": "RECEIPT",
    "amount_minor": 5000000,
    "currency": "BDT",
    "reference": "CHQ-88219",
    "reverses_entry_id": null,
    "note": "Initial assessment fee received",
    "created_at": "2026-10-15T11:00:00Z"
  }
]
```

---

4. OCR & Local LLM Integration

4.1 Google Cloud Vision OCR Engine (`app/ocr_engine.py`)

```
from google.cloud import vision
import io, json, os

class GoogleOcrEngine:
    def __init__(self, credentials_path=None):
        if credentials_path:
            os.environ['GOOGLE_APPLICATION_CREDENTIALS'] = credentials_path
        self.client = vision.ImageAnnotatorClient()

    def extract_page_text(self, image_bytes: bytes) -> str:
        image = vision.Image(content=image_bytes)
        image_context = vision.ImageContext(language_hints=['bn', 'en'])
        response = self.client.document_text_detection(image=image, image_context=image_context)
        if response.error.message:
            raise RuntimeError(f"Google OCR Error: {response.error.message}")
        return response.full_text_annotation.text
```

4.2 Local LangChain LLM Client (`app/llm_engine.py`)

```
from langchain_community.chat_models import ChatOllama
from langchain_core.messages import SystemMessage, HumanMessage
import json

class LocalBanglaLLMEngine:
    def __init__(self, model_name="qwen2.5:7b-instruct", base_url="http://localhost:11434"):
```

```
        self.llm = ChatOllama(
            model=model_name,
            base_url=base_url,
            temperature=0.0,
            format="json"
        )

    def extract_candidates(self, system_prompt: str, document_text: str) -> dict:
        messages = [
            SystemMessage(content=system_prompt),
            HumanMessage(content=f"<DOCUMENT>\n{document_text}\n</DOCUMENT>")
        ]
        response = self.llm.invoke(messages)
        return json.loads(response.content)
```

---

5. Security & Deployment

- **Network Containment:** Operated within trusted localhost or isolated VPC networks.
- **Zero Cloud Leakage:** All source files, evidence, and inference remain on the local machine except for OCR calls to Google Vision.
- **Production Command:** Run using Gunicorn with 2 workers and 4 threads:

```
gunicorn -w 2 --threads 4 -b 0.0.0.0:5000 run:app
```

---

6. Explained with Diagrams & Workflows

6.1 Overview

TenderOS Lite is a private, automated software system designed to help teams quickly process, review, and evaluate complex tender documents. It automatically reads uploaded tender papers in Bengali and English, identifies key eligibility criteria and requirements using artificial intelligence, and provides secure tracking for financial records and team decisions without leaking confidential data outside your network.

6.2 Visual System Architecture

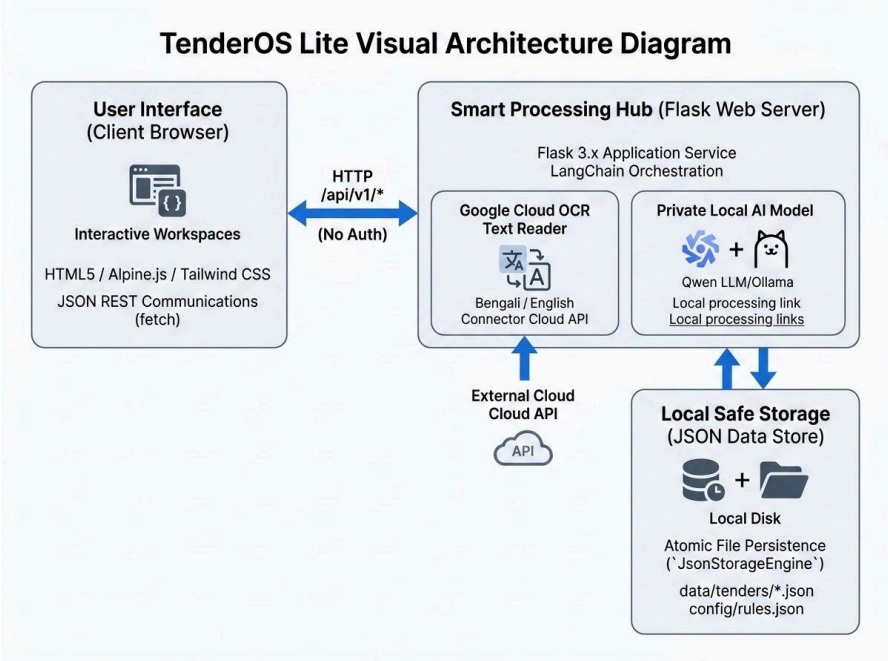


Figure 6.1: High-level system architecture showing data flow from user browser to processing engines and local storage.

6.3 Step-by-Step Document Processing Workflow

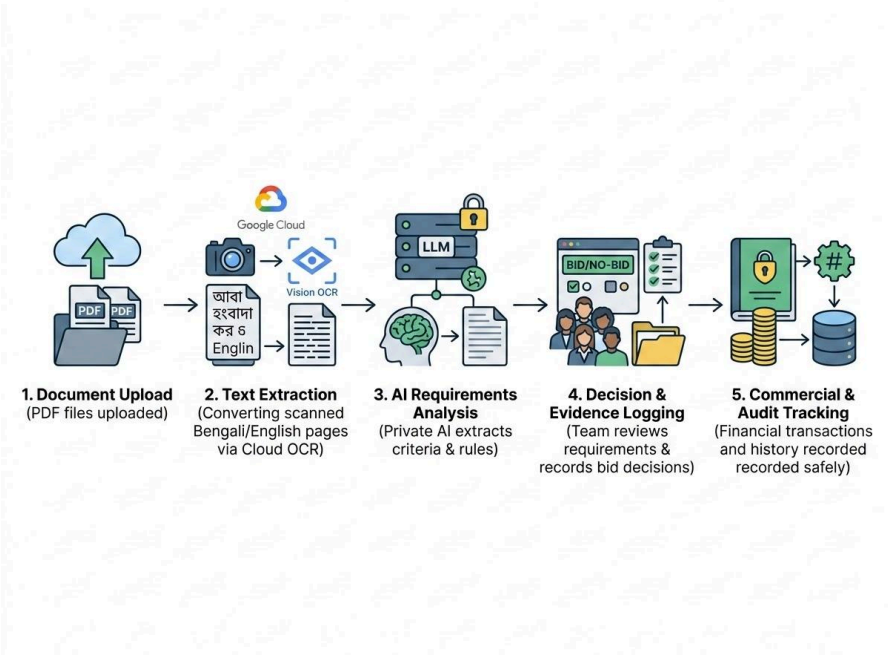


Figure 6.2: End-to-end 5-step workflow for processing tender proposals.

6.4 Key Benefits & Technical Terminology Guide

- **Local LLM (Private Local AI):** An artificial intelligence model that runs entirely on your local server. It analyzes document contents without sending proprietary information to third-party public AI platforms.
- **OCR (Image-to-Text Reader):** Optical Character Recognition technology that converts scanned PDF documents or images into searchable and editable digital text in both Bengali and English.

- **JSON Storage (Structured File Folders):** A lightweight, highly organized system for saving records directly as structured text files, avoiding complex database maintenance while ensuring safe data updates.

- **Audit Ledger (Transaction History):** A tamper-proof history log that keeps record of all actions, document changes, and financial entries for complete transparency and compliance.

TenderOS Lite — Software Requirements Specification (SRS)

Document ID: TOS-SRS-LITE-1.0

Project: TenderOS Simplified Architecture (TenderOS Lite)

Target Environment: Local / Production-Ready Appliance (Zero-Auth / Trusted Intranet)

Technology Stack: Python 3.11+ / Flask / LangChain / JSON File-Based Storage / Google Cloud Vision OCR / Local Lightweight LLM (Bangla-capable) / Alpine.js + Tailwind CSS

---

1. Executive Summary & Philosophy

TenderOS Lite simplifies the procurement readiness workbench architecture while retaining 100% of its core operational features and evidentiary discipline. All authentication, user management, and session gatekeeping have been eliminated in favor of an immediate-access, zero-friction operator workflow.

Complex external database dependencies (MongoDB, PostgreSQL, SQLite triggers) are replaced with a **structured JSON file-based document and ledger engine**. External text extraction pipelines are upgraded with **Google Cloud Vision OCR** (supporting Bengali and English script recognition on scanned PDFs) combined with a **local, lightweight, open-source LLM runtime** (such as Qwen2.5-7B-Instruct or Llama-3.2-3B via llama.cpp/Ollama) to extract tender requirement candidates and compute compliance scores locally.

Rules, validation gates, and workflow criteria are maintained in an externalized, human-auditable `tenderos_rules.json` configuration file, ensuring complete traceability and client-ready governance.

---

2. Core Functional Requirements

2.1 FR-SYS: System Access & Session Management

- **FR-SYS-1 (M): Direct Operator Access:** The system shall not require logins, passwords, MFA, JWTs, or role checks. Launching the backend service and navigating to `http://localhost:5000` shall grant full access to all tender evaluation, review, and reporting interfaces.
- **FR-SYS-2 (M): Single Operator Persona:** All interactions are executed under a default system persona (`Operator`), recorded in transaction histories as `SYSTEM_OPERATOR`.

2.2 FR-SRC: Ingestion & Dual OCR Pipeline

- **FR-SRC-1 (M): PDF File Validation & Storing:** Ingest source tender notices, schedules, and amendments. Compute the SHA-256 digest of original PDF bytes and store files under `data/tenders/{tender_id}/sources/{file_sha256}.pdf`.
- **FR-SRC-2 (M): Native Digital PDF Text Parsing:** For digital documents with accessible text streams, parse directly via `pypdf` on a per-page boundary.
- **FR-SRC-3 (M): Google Cloud Vision OCR Integration:** Scanned or image-only pages (flagged by character density < 40 characters/page) shall automatically route to Google Cloud Vision OCR (`images:annotate` API with language hint `['bn', 'en']`). Output text, bounding blocks, and confidence scores are cached under `data/tenders/{tender_id}/ocr/{source_id}_p{page}.json`.

2.3 FR-ANA: LangChain Bounded Extraction Pipeline

- **FR-ANA-1 (M): Five-Stage Deterministic Pipeline:**

1. `SOURCE_INSPECT`: Page integrity, text/OCR resolution, and quality flags.
2. `CHUNK_EMBED`: Chunking (800 chars, 100 char overlap) using LangChain `RecursiveCharacterTextSplitter`.
3. `REQUIREMENT_CANDIDATES`: Structured requirement extraction using local open-source LLM.
4. `EVIDENCE_RELEVANCE`: Token overlap and lexical match between requirements and private evidence.
5. `CITATION_CHECK`: Exact string and fuzzy Levenshtein re-verification against source text.

- **FR-ANA-2 (M): Local Lightweight LLM Integration:** LangChain `ChatOllama` / `ChatOpenAI` (OpenAI-compatible local server) executing local GGUF weights (e.g., Qwen2.5-7B or Llama-3.2-3B) with explicit Bengali language support.
- **FR-ANA-3 (M): Enforced JSON Output:** LLM outputs are strictly constrained to JSON schemas via LangChain Pydantic output parsers or guided JSON schema output flags. Unparseable outputs trigger a single retry before falling back to rule-based keyword extraction.

2.4 FR-RUL: Externalized JSON Rules Engine

- **FR-RUL-1 (M): External Rule Specification:** All extraction thresholds, category classifications, decision gates, and currency expiration limits shall be loaded dynamically from `config/tenderos_rules.json`.
- **FR-RUL-2 (M): Dynamic Rule Updates:** Modifying `tenderos_rules.json` applies immediately on subsequent runs without requiring code redeployment.

2.5 FR-REV: Review Queue & Human Gatekeeping

- **FR-REV-1 (M): Unified Review Worklist:** Centralized view of unresolved candidate requirements, missing source inventory, expired evidence, and stale decisions.
- **FR-REV-2 (M): Human Gatekeeping:** Candidate acceptance creates unreviewed, `UNKNOWN` requirements. Candidates lacking `CITATION_VALID` verification cannot be accepted.

2.6 FR-DEC: Human Procurement Decisions

- **FR-DEC-1 (M): BID / NO_BID / HOLD Recording:** Operators record procurement decisions with a mandatory rationale (minimum 10 characters).
- **FR-DEC-2 (M): Staleness Detection:** Adding a source addendum, modifying a requirement, or updating an evidence file automatically marks existing decisions as `STALE`.

2.7 FR-COM: Commercial Ledger & Diagnostic Releases

- **FR-COM-1 (M): Append-Only Ledger:** Financial transactions (`RECEIPT`, `DIRECT_COST`, `REFUND`, `REVERSAL`) recorded in BDT integer minor units (paisa). Modification or deletion of historical records is blocked.
- **FR-COM-2 (M): Frozen Diagnostic Releases:** Generates immutable diagnostic snapshots saved under `data/tenders/{tender_id}/releases/v{n}.json` and exportable to clean DOCX/CSV.

---

3. Data Integrity & File-Based Persistence

All data entities are stored in structured JSON files on disk under `data/`:

```
data/
├── tenders/
```

```
│   └── {tender_id}/
│       ├── tender.json
│       ├── inventory.json
│       ├── requirements.json
│       ├── evidence.json
│       ├── decisions.json
│       ├── ledger.json
│       ├── runs/
│       │   └── {run_id}.json
│       ├── releases/
│       │   └── release_v{n}.json
│       └── sources/
│           └── {file_sha256}.pdf
└── audit/
    └── system_audit.jsonl
```

Atomic file operations (`tempfile` + `os.replace`) ensure single-process write safety and zero corruption risk.

Project file v1 : tenderos-lite.zip

TenderOS V3 — How It Works

The Complete & Professional Guide to Automated Bid Processing

1. Executive Summary & Overview

Traditional procurement processes require manually reviewing complex, multi-page tender documents—a task that often consumes days and increases the risk of costly oversights or missing mandatory compliance criteria.

TenderOS acts as an end-to-end intelligent assistant for proposal creation and tender analysis. By automating ingestion, language processing, and data extraction, TenderOS drastically reduces preparation time from days to minutes while ensuring full compliance with technical and administrative requirements.

Key Capabilities at a Glance

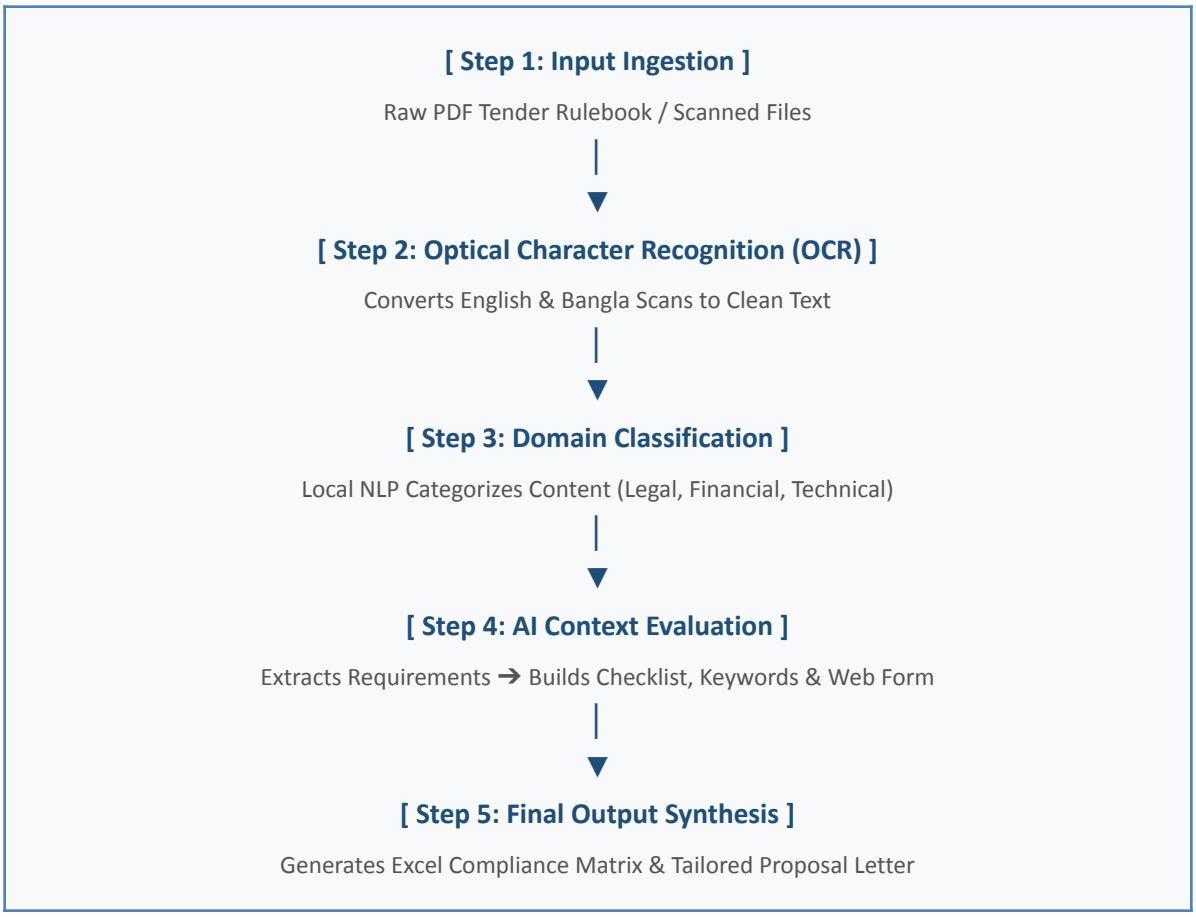
- **Rapid Ingestion:** Parses large PDF rulebooks and extracts critical parameters in under 30 seconds.
- **Bilingual Support:** Handles complex mixed-language documents seamlessly, including English and Bangla.
- **Compliance Guarantee:** Automatically flags required certificates, financial prerequisites, and legal papers.
- **Automated Drafting:** Generates structured compliance checklists, key evaluation terms, and interactive forms for customized proposal output.

2. The 5-Step Workflow Architecture

Step Number	What We Call It	What Actually Happens	Why It Is Awesome
Step 1	**Drag & Drop**	Upload raw PDF tender documents via a drag-and-drop web interface without technical setup.	Instant initiation; frictionless entry point for all team members.
Step 2	**Multilingual Optical Character Recognition (OCR)**	OCR processing scans physical pages, digital PDFs, and low-resolution scans across English and Bangla scripts.	Converts non-searchable or degraded scans into accurate digital text.
Step 3	**Local Context Categorization (Local NLP)**	Natural language processing sorts raw text into structured operational domains (Legal, Financial,	Pre-structures data to streamline downstream AI analysis and prevent context dilution.

		Technical, General Rules).	
Step 4	**Deep AI Evaluation & Extraction**	Advanced LLM models analyze sorted text to generate compliance checklists, target evaluation terms, and query forms.	Pinpoints mandatory prerequisites and highlights high-weighting bid criteria.
Step 5	**Artifact Generation & Submission**	Compiles an organized Excel tracking ledger and drafts a tailored, professional proposal response letter.	Delivers fully formatted, error-free submission packages in minimal turnaround time.

System Processing Flowchart



3. Core Output Tools & Deliverables

Tool 1: Mandatory Compliance Checklist

Systematically extracts every required submission document (such as trade licenses, tax audit reports, and bank solvency certificates). Ensures no mandatory paper is overlooked prior to bid submission.

Tool 2: Key Evaluation Terms & Magic Keywords

Identifies scoring terminology and technical criteria hidden within complex tender specifications. Incorporating these exact keywords maximizes technical scoring during evaluation.

Tool 3: Interactive Dynamic Form & Proposal Draft

Presents clean, structured web input fields for missing vendor-specific data. Once filled, TenderOS auto-populates all necessary details to compile a fully formatted proposal letter.

Summary Matrix of Deliverables

Deliverable Tool	Primary Functionality	Strategic Benefit
1. Document Checklist	Aggregates and verifies administrative and statutory requirements.	Eliminates disqualifications due to missing supporting documents.
2. Key Evaluation Keywords	Extracts scoring terms and specific technical requirements.	Increases technical evaluation marks by aligning language directly with scoring criteria.
3. Dynamic Input Form	Collects targeted operational variables from the user team.	Streams organizational inputs into finalized proposal letters and tracking ledgers.

4. Comparative Impact Analysis

Evaluation Dimension	Traditional Manual Process	Automated TenderOS Workflow
Processing Time	Manual reading and review: 2–3 business days.	AI analysis complete in under 30 seconds.
Compliance Accuracy	High risk of missed administrative requirements.	100% automated extraction and verification checklist.
Proposal Quality	Generic responses with potential keyword gaps.	Precision targeting using extracted evaluation terms.
Data Collection	Disjointed email threads and manual document assembly.	Centralized dynamic form input and automated document compilation.

TenderOS V3 — Product Requirements Document (PRD)

Document ID: TOS-PRD-V3.0

Project: TenderOS Redesign (Automated Proposal & Checklist Engine)

Version: 3.0 Production Blueprint

Target Users: Tender Consultants, Bid Managers, B2B Contractors (Bangladesh e-GP & International)

---

1. Product Vision & Value Proposition

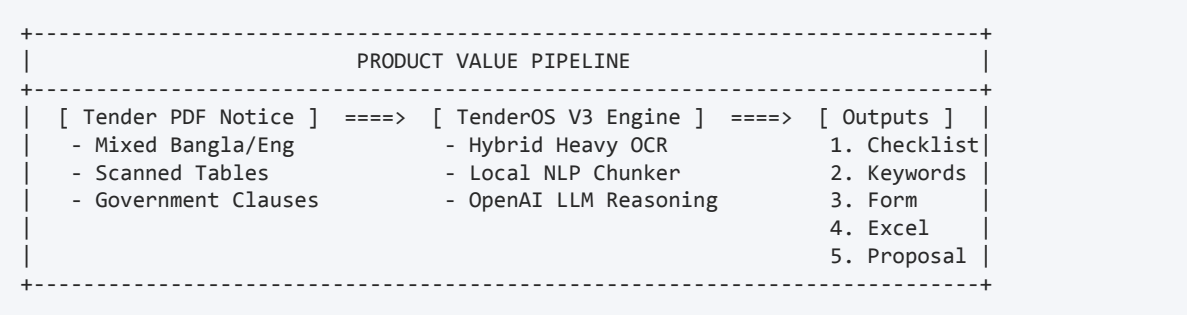
TenderOS V3 shifts from a passive document viewer into an **active generation pipeline**. Instead of only showing red flags, the platform ingests complex tender PDF notices (in mixed Bengali and English), parses them, and immediately yields:

1. An actionable **Document Checklist** so teams never miss an eligibility certificate.

2. A **Proposal Keyword Matrix** ensuring the bid letter mirrors the evaluation criteria.

3. A **Dynamic Client Intake Form** to collect required organizational facts from clients.

4. An instantly downloadable **Excel Workbook** and synthesized **Proposal Document**.



---

2. User Journey & Step-by-Step Experience

Step 1: Zero-Friction Tender Upload

- The user visits the website; no login or password is required.
- The user drags and drops one or more tender PDFs (Main Notice, Technical Specifications, Addenda).

- Immediate confirmation display with file size, page count, and SHA-256 hash.

Step 2: Automated Hybrid Extraction & Analysis

- Behind the scenes, the system executes heavy parsing and Bengali/English OCR.
- The local NLP normalizer segments clauses into clean JSON.
- LangChain calls OpenAI to generate structured outputs.
- A visual progress stepper shows:

```
[Files Uploaded] -> [OCR & Parsing] -> [Clause Normalization] -> [AI Extraction] -> [Ready]
```

Step 3: Interactive Results & Review Workspace

- **Review Tab:** Displays the extracted document checklist with status toggles.
- **Keywords Tab:** Shows high-scoring keywords with context quotes and copy-to-clipboard functionality.
- **Excel Download:** One-click download button for `tender_requirements.xlsx`.

Step 4: Client Intake Form Generation

- The system generates an interactive web form based on `client_form_schema.json`.
- The user (or their client) fills in specific bid parameters (e.g., turnover, past project experience, bank solvency amounts).

Step 5: Proposal Generation

- Clicking "Generate Proposal" combines the tender requirements, client inputs, and keyword matrix into a professional `.docx` proposal letter.

---

3. Product Feature Matrix

Feature	Description	Business Impact
Zero-Auth Access	Instant access on internal networks without credentials	Zero friction; immediate team productivity
Heavy Bengali/English OCR	Reads scanned government PDFs with local OCR	Solves unsearchable e-GP scan bottlenecks
Local NLP Normalizer	Chunks raw pages into structured JSON clauses	Cuts cloud LLM token costs by 60%
Document Checklist	Itemized list of mandatory licenses and forms	Prevents disqualification due to missing documents

Proposal Keyword Matrix	Essential technical terms required in the proposal	Maximizes technical evaluation scoring
Dynamic Form Generator	Auto-generates client intake form from tender specs	Eliminates manual questionnaires
Excel Requirements Sheet	Clean multi-tab `.xlsx` file for offline teams	Standard working format for bidding teams
Proposal Synthesis	Assembles compliant bid letters automatically	Cuts bid preparation time from days to hours

---

4. Key Performance Targets

- **Parsing & Extraction Speed:** < 60 seconds for a 50-page mixed PDF.
- **Checklist Accuracy:** 98% recall on mandatory qualification documents.
- **Keyword Coverage:** 100% extraction of weighted technical criteria.
- **Form Usability:** Instant zero-code rendering of client input forms.

TenderOS V3 — Software Requirements Specification (SRS)

Document ID: TOS-SRS-V3.0

Project: TenderOS Redesign (Automated Proposal & Checklist Engine)

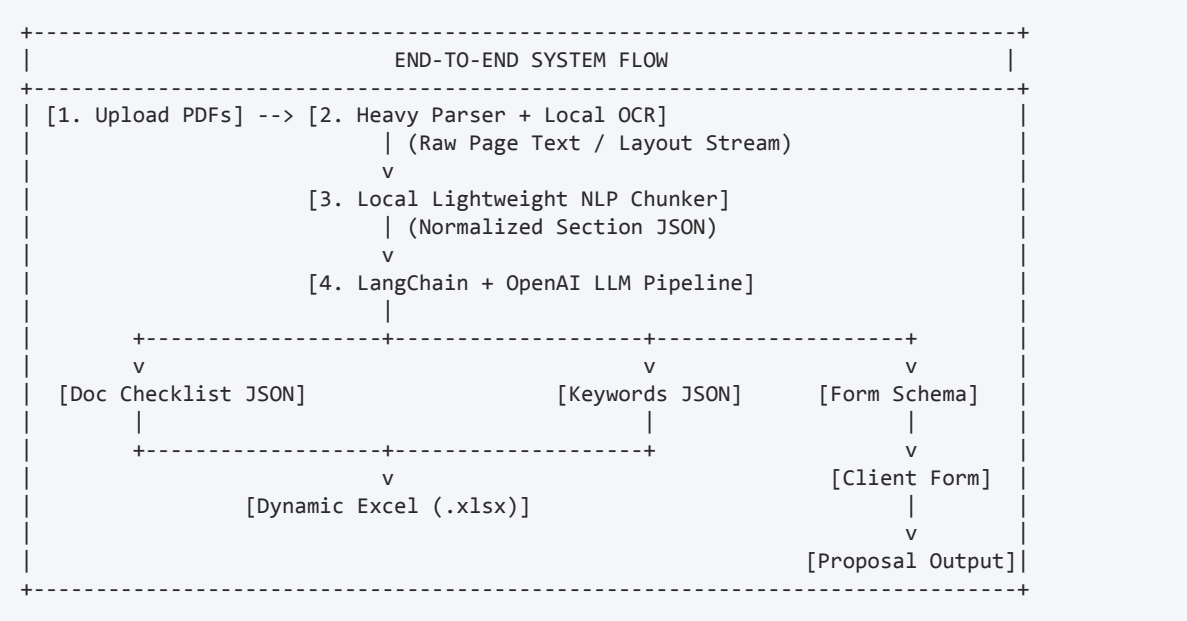
Environment: Zero-Auth Operator Web Appliance / Intranet

Stack: Flask 3.x / LangChain 0.3 / OpenAI (GPT-4o/GPT-4o-mini) / Local Heavy Bengali-English Parser + Local OCR / JSON Storage / openpyxl / Alpine.js + Tailwind CSS

---

1. System Scope & Engineering Reality Check

This specification redesigns the TenderOS core into an end-to-end tender preparation, client dynamic form generation, and proposal generation engine.



Brutal Engineering Review & Constraints

1. **The "Local NLP to JSON" Bottleneck:** Raw extracted text from messy 200-page government tender documents cannot simply be handed to a "small NLP" model to create an arbitrary JSON representation without severe data truncation. The local NLP stage must function as a **Deterministic Structural Normalizer & Chunker** (segmenting ITT, TDS, GCC, PCC, and tabular schedules into bounded, indexed JSON sections), preventing token loss before reaching the OpenAI pipeline.

2. **Deterministic Schema Contract:** The OpenAI LLM outputs must be governed by strict Pydantic JSON schemas. If the model hallucinates formatting, dynamic form generation and Excel generation will crash.

3. **Open Form Pipeline:** The dynamic form is rendered on the client browser using a standard JSON Schema (`form_schema.json`), collecting company-specific parameters, and feeding back to construct the customized bid proposal.

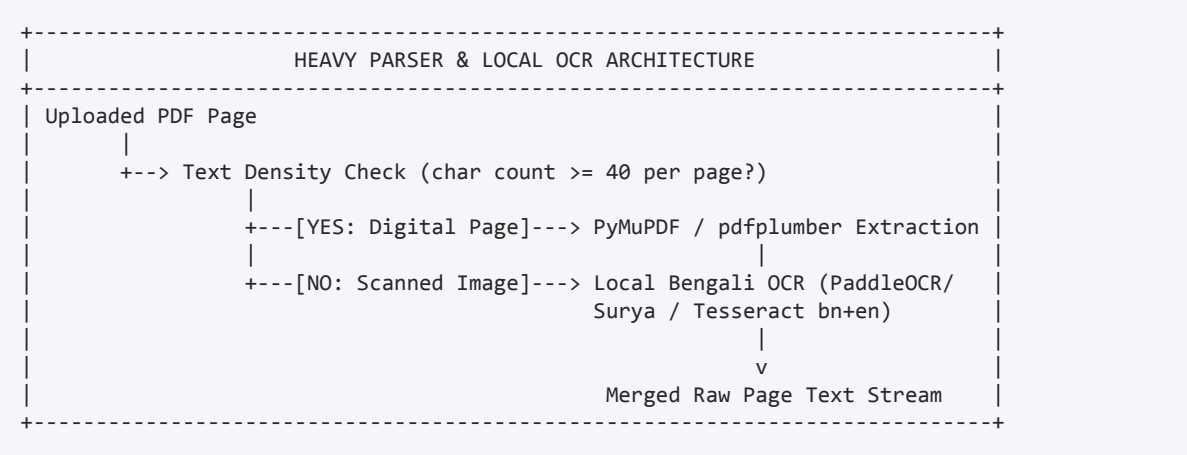
---

2. Functional Requirements

2.1 FR-ING: Tender Document Ingestion

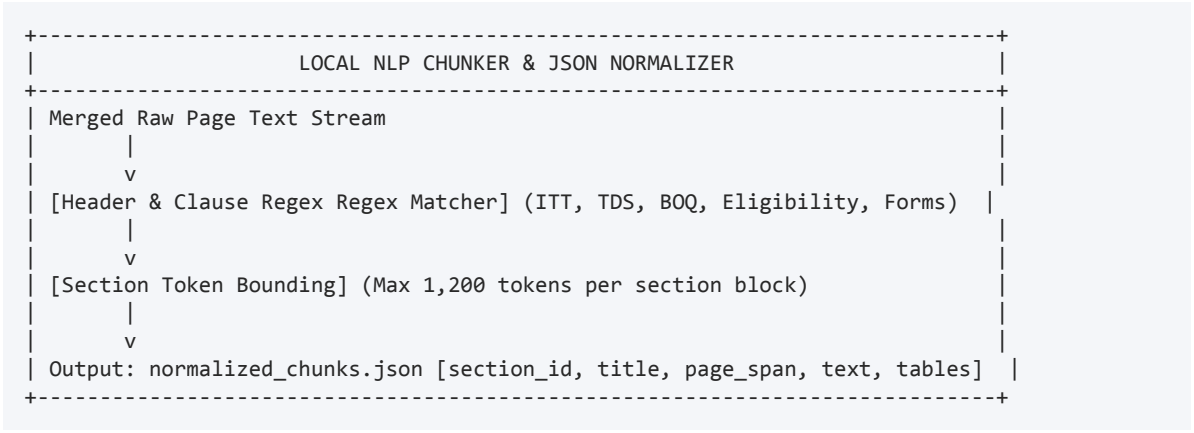
- **FR-ING-1 (M):** Multi-file PDF upload (Notices, Schedules, BOQs, Addenda) without authentication or login barriers.
- **FR-ING-2 (M):** Compute and record SHA-256 digest per uploaded PDF to guarantee evidentiary immutability.
- **FR-ING-3 (M):** Maintain an inventory manifest linking parent notices and subsequent amendments.

2.2 FR-PRS: Heavy Dual-Stream Parser & Local OCR



- **FR-PRS-1 (M):** Inspect each page using a hybrid text density evaluator (threshold = 40 characters).
- **FR-PRS-2 (M):** Digital pages are parsed using `PyMuPDF` (fitz) or `pdfplumber` to preserve tables and character layout.
- **FR-PRS-3 (M):** Scanned/image pages are processed via a local OCR engine supporting Bengali script (PaddleOCR Bengali or Tesseract `ben+eng`), yielding coordinates, text, and confidence scores.

2.3 FR-NLP: Local Lightweight NLP Structuring



- **FR-NLP-1 (M):** Ingest raw page text streams and segment them into functional procurement clauses using keyword regex and layout analysis.

- **FR-NLP-2 (M):** Construct `normalized_chunks.json` grouping clauses by topic: Eligibility, Technical Compliance, Financial Capacity, Required Forms, and Submission Timelines.

2.4 FR-LLM: LangChain OpenAI Extraction Engine

- **FR-LLM-1 (M):** Execute LangChain `RunnableSequence` calling OpenAI (e.g., `gpt-4o-mini` or `gpt-4o`) with strict Pydantic JSON schema constraints.

- **FR-LLM-2 (M):** Output 1: `doc_checklist.json` — Categorized inventory of every certificate, license, financial statement, and litigation record required.

- **FR-LLM-3 (M):** Output 2: `proposal_keywords.json` — High-weight technical and commercial terminology extracted directly from tender clauses for inclusion in the proposal.

- **FR-LLM-4 (M):** Output 3: `client_form_schema.json` — A dynamic JSON form specification containing all required client inputs (turnover numbers, experience references, bank facilities, litigation declarations).

2.5 FR-XLS: Automated Excel Workbook Generator

- **FR-XLS-1 (M):** Produce an automated multi-tab Excel workbook (`.xlsx`) via `openpyxl`:

- **Tab 1: Document Checklist** (Document Name, Clause Ref, Mandatory/Optional, Issuing Authority, Verification Status).

- **Tab 2: Proposal Keywords** (Keyword/Phrase, Tender Section, Context Quote, Recommended Placement in Proposal).

- **Tab 3: Client Data Form** (Field Name, Description, Source Clause, Required Format).

2.6 FR-FRM: Dynamic Client Form & Proposal Synthesis

- **FR-FRM-1 (M):** Web interface dynamically parses `client_form_schema.json` into an interactive client intake form.

- **FR-FRM-2 (M):** On form submission, client answers are validated against the schema and stored in `client_submission.json`.
- **FR-FRM-3 (M):** Assemble the final proposal document combining the client data, extracted requirements, and required proposal keywords.

---

3. Data Flow & Interface Contracts

Artifact Name	Generating Component	Format	Purpose
`sources.json`	Ingestion Blueprint	JSON	Metadata, SHA-256 hashes, file locations
`normalized_chunks.json`	Local NLP Chunker	JSON	Clustered clauses and tables with page references
`doc_checklist.json`	LangChain + OpenAI	JSON	Machine-verifiable list of mandatory documents
`proposal_keywords.json`	LangChain + OpenAI	JSON	Keywords and phrases required in the bid letter
`client_form_schema.json`	LangChain + OpenAI	JSON	Dynamic form definition for client data collection
`tender_requirements.xlsx`	Excel Generator	XLSX	Downloadable spreadsheet for procurement teams
`client_submission.json`	Client Form UI	JSON	Actual client inputs submitted through the web form
`final_proposal.docx`	Proposal Generator	DOCX	Fully synthesized tender proposal letter

TenderOS V3 — Technical Requirements & Design Document (TRD)

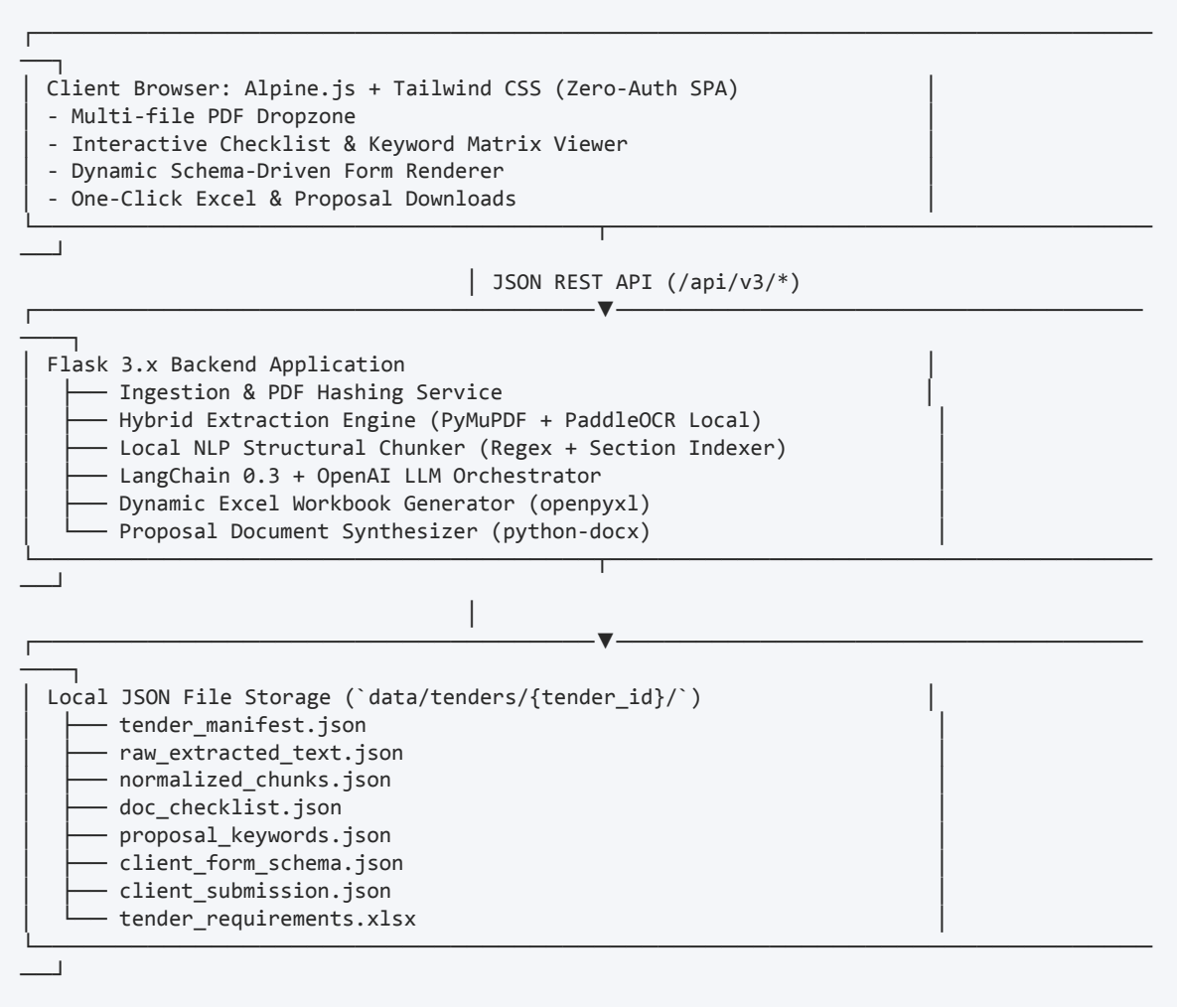
Document ID: TOS-TRD-V3.0

Project: TenderOS Redesign (Automated Proposal & Checklist Engine)

Implementation Reference: Flask 3.x / LangChain 0.3 / OpenAI API / PaddleOCR & PyMuPDF / openpyxl / Alpine.js

---

1. Technical Architecture & Component Interaction



---

2. Detailed Technical Subsystems

2.1 Ingestion & Hybrid Extraction Pipeline

```
# app/services/extraction.py
import fitz  # PyMuPDF
from paddleocr import PaddleOCR
import hashlib, os, json

class HybridExtractor:
    def __init__(self):
        # Bengali + English local OCR engine
        self.ocr = PaddleOCR(use_angle_cls=True, lang='bengali')

    def process_pdf(self, pdf_path: str) -> list[dict]:
        doc = fitz.open(pdf_path)
        pages_output = []
        for page_num in range(len(doc)):
            page = doc[page_num]
            text = page.get_text()
            if len(text.strip()) >= 40:
                pages_output.append({
                    "page": page_num + 1,
                    "method": "digital_pymupdf",
                    "text": text
                })
            else:
                pix = page.get_pixmap()
                img_path = f"/tmp/page_{page_num}.png"
                pix.save(img_path)
                ocr_result = self.ocr.ocr(img_path, cls=True)
                ocr_text = "\n".join([line[1][0] for block in ocr_result for line in
block])
                pages_output.append({
                    "page": page_num + 1,
                    "method": "local_ocr_bengali",
                    "text": ocr_text
                })
                if os.path.exists(img_path): os.remove(img_path)
        return pages_output
```

2.2 Local NLP Normalizer & Section Chunker

```
# app/services/nlp_chunker.py
import re, json

class LocalNLPChunker:
    PATTERNS = {
        "eligibility": r"(eligibility|qualification|experience|turnover|যোগ্যতা|অভিজ্ঞতা)",
        "technical": r"(technical specification|scope of work|কারিগরি বিবরণী)",
        "forms": r"(standard form|submission form|tender submission letter|ফরম)"
    }

    def chunk_document(self, pages: list[dict]) -> list[dict]:
        chunks = []
        current_chunk = {"category": "general", "text": "", "pages": []}
        for p in pages:
            text = p["text"]
            for cat, pattern in self.PATTERNS.items():
                if re.search(pattern, text, re.IGNORECASE):
                    if current_chunk["text"]:
                        chunks.append(current_chunk)
                    current_chunk = {"category": cat, "text": text, "pages": [p["page"]]}
                    break
            else:
                current_chunk["text"] += "\n" + text
                current_chunk["pages"].append(p["page"])
        if current_chunk["text"]:
            chunks.append(current_chunk)
        return chunks
```

2.3 LangChain OpenAI Pipeline

```
# app/services/llm_pipeline.py
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel, Field
import json

class DocumentItem(BaseModel):
    name: str = Field(description="Name of the required document or certificate")
    clause_reference: str = Field(description="Tender clause reference or page number")
    mandatory: bool = Field(description="True if mandatory, false if optional")
    description: str = Field(description="Details on issuing authority or validity")

class KeywordItem(BaseModel):
    keyword: str = Field(description="Key phrase or terminology")
    context: str = Field(description="Clause excerpt demonstrating usage")
    section: str = Field(description="Target section in the proposal letter")

class FormField(BaseModel):
    field_id: str
    label: str
    input_type: str = Field(description="text, number, date, or file")
    required: bool
    description: str

class TenderAnalysisOutput(BaseModel):
    checklist: list[DocumentItem]
    keywords: list[KeywordItem]
    form_schema: list[FormField]

class TenderLLMService:
    def __init__(self, api_key: str):
        self.llm = ChatOpenAI(model="gpt-4o", api_key=api_key, temperature=0.0)

    def analyze(self, normalized_json_str: str) -> dict:
        prompt = ChatPromptTemplate.from_messages([
            ("system", "You are an expert tender bid director. Analyze the normalized
tender clauses and extract: 1) Document checklist, 2) Proposal keywords, and 3) Client
intake form schema."),
            ("human", "TENDER CLAUSES:\n{clauses}")
        ])
        chain = prompt | self.llm.with_structured_output(TenderAnalysisOutput)
        result = chain.invoke({"clauses": normalized_json_str})
        return result.model_dump()
```

2.4 Excel Generator (`openpyxl`)

```
# app/services/excel_exporter.py
import openpyxl
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side

def generate_tender_excel(checklist: list, keywords: list, form_schema: list, filepath:
str):
    wb = openpyxl.Workbook()
    ws_check = wb.active
    ws_check.title = "Document Checklist"
    ws_key = wb.create_sheet("Proposal Keywords")
    ws_form = wb.create_sheet("Client Form Requirements")

    # Header styling
    header_fill = PatternFill(start_color="1F4E79", end_color="1F4E79", fill_type="solid")
    header_font = Font(name="Calibri", size=11, bold=True, color="FFFFFF")

    # Tab 1: Checklist
    ws_check.append(["Document Name", "Clause Reference", "Mandatory", "Description"])
    for row in ws_check[1]: row.fill = header_fill; row.font = header_font
    for doc in checklist:
        ws_check.append([doc["name"], doc["clause_reference"], "YES" if doc["mandatory"]
else "NO", doc["description"]])

    # Tab 2: Keywords
    ws_key.append(["Keyword / Term", "Section", "Context Quote"])
    for row in ws_key[1]: row.fill = header_fill; row.font = header_font
    for k in keywords:
        ws_key.append([k["keyword"], k["section"], k["context"]])

    # Tab 3: Form Fields
    ws_form.append(["Field Label", "Input Type", "Required", "Description"])
    for row in ws_form[1]: row.fill = header_fill; row.font = header_font
    for f in form_schema:
        ws_form.append([f["label"], f["input_type"], "YES" if f["required"] else "NO",
f["description"]])

    wb.save(filepath)
```

---

3. Directory Layout & JSON Contracts

```
data/tenders/{tender_id}/
├── tender_manifest.json          # Overall state, file hashes, timestamps
├── raw_extracted_text.json       # Output from PyMuPDF + PaddleOCR
├── normalized_chunks.json        # Output from Local NLP Chunker
├── doc_checklist.json            # Pydantic parsed checklist
├── proposal_keywords.json        # Pydantic parsed keywords
├── client_form_schema.json       # Pydantic parsed dynamic form schema
├── client_submission.json        # User submitted responses from dynamic form
├── tender_requirements.xlsx      # Generated 3-tab Excel workbook
└── generated_proposal.docx       # Final synthesized proposal letter
```